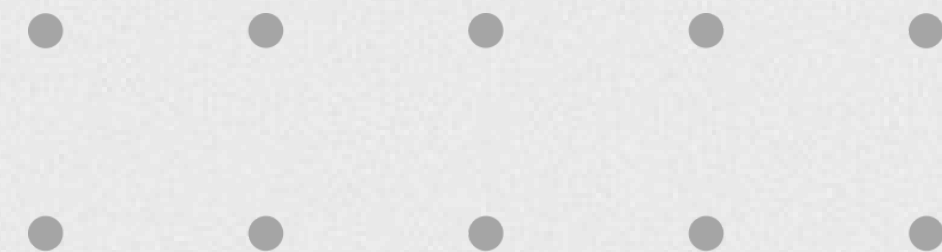
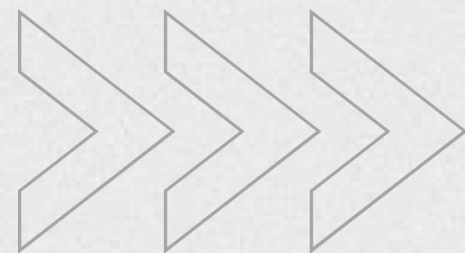


SISTEMA DE IDENTIFICAÇÃO VISUAL E CONTAGEM DE PEÇAS COM VISÃO COMPUTACIONAL E COMUNICAÇÃO MODBUS/TCP, S7 E MQTT

Docente: *Prof. Me. Tiago Prati*

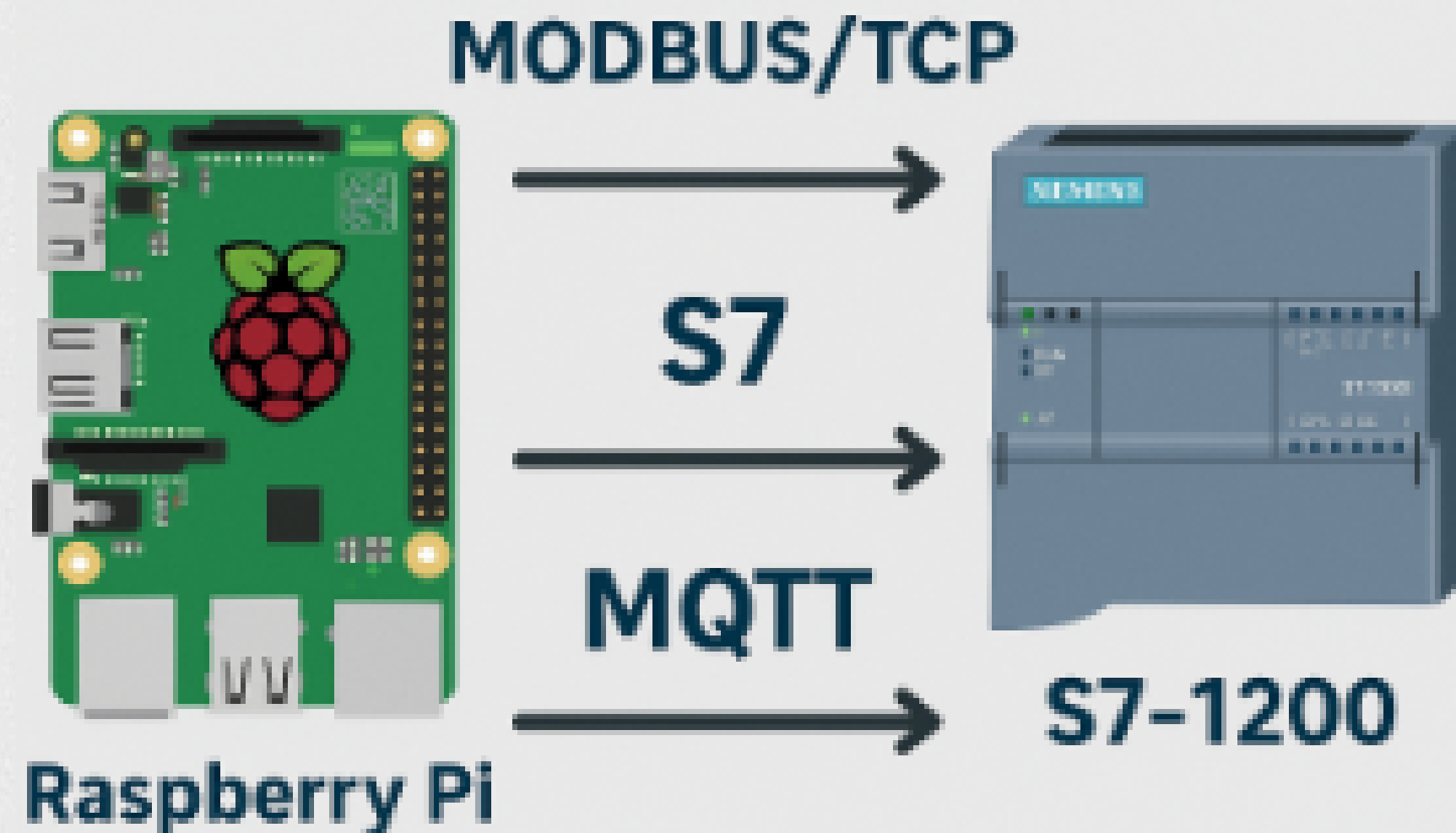
Discentes: *Lucas Boarim Paludo e Lucas Ursulino*



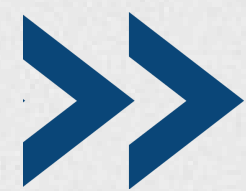


Introdução

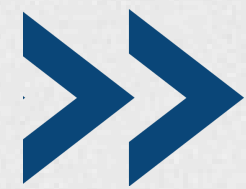
Desenvolvimento de um sistema que utiliza visão computacional para reconhecimento e contagem de peças, com comunicação via rede através de protocolos industriais.



Objetivos

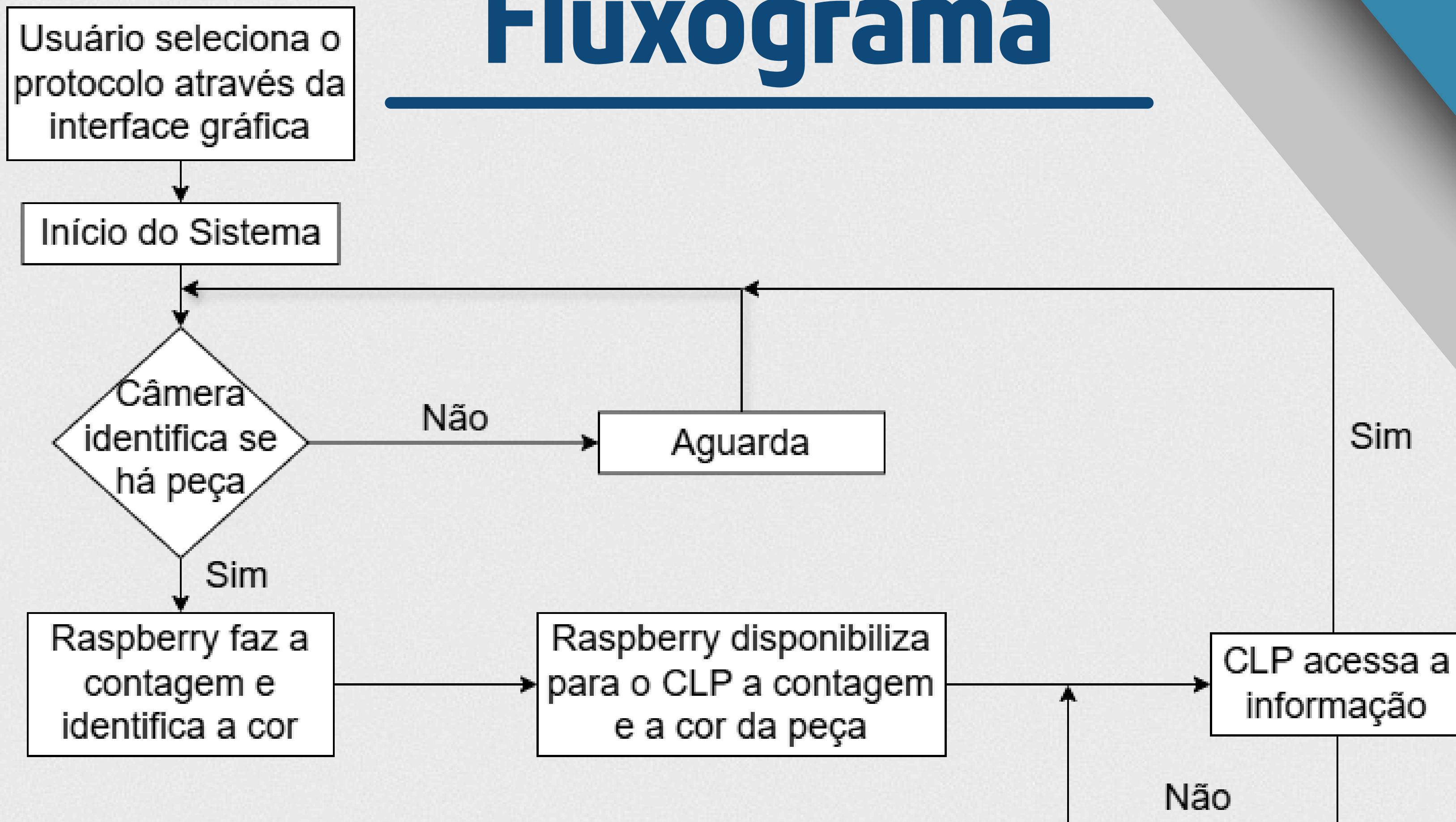


Identificação visual de peças vermelhas, pretas, cromadas e transparentes.



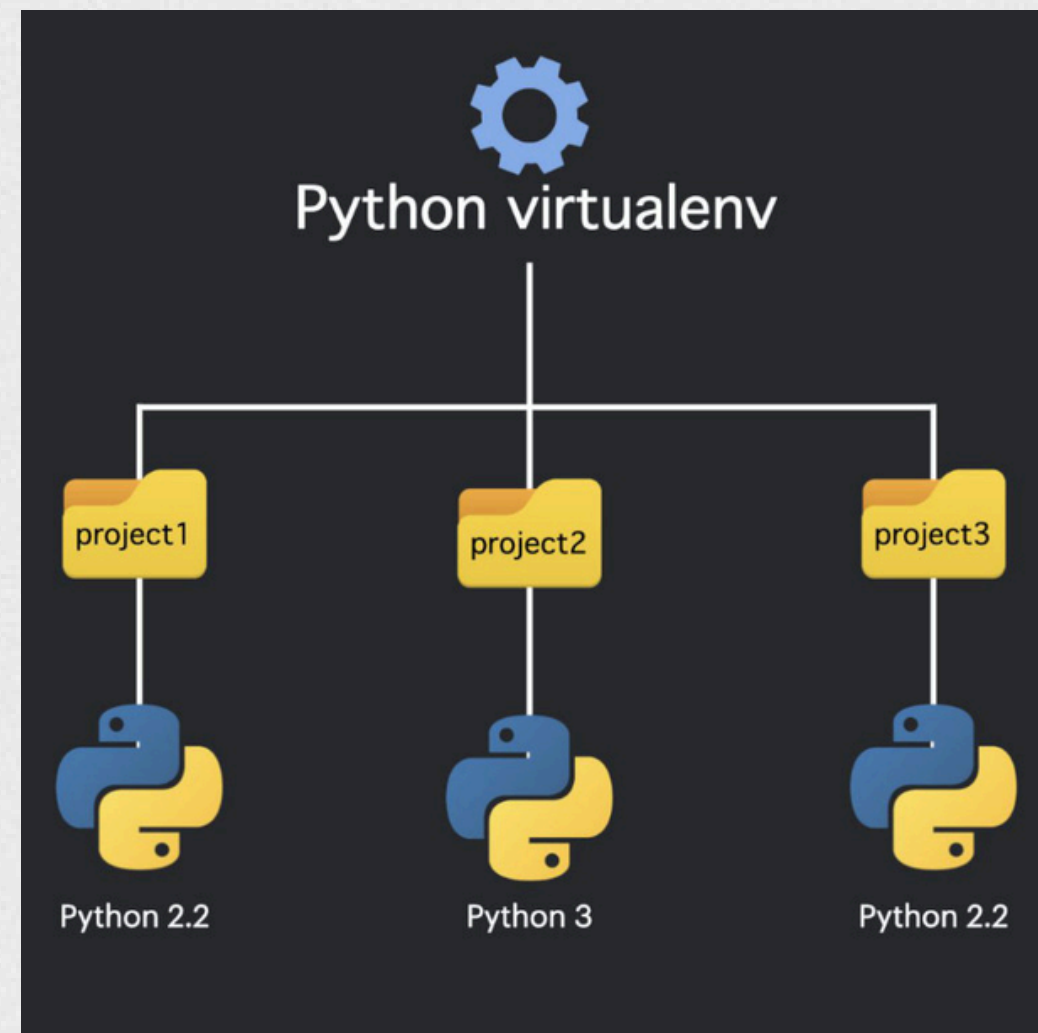
Integração com os protocolos MODBUS/TCP, S7 e MQTT.

Fluxograma

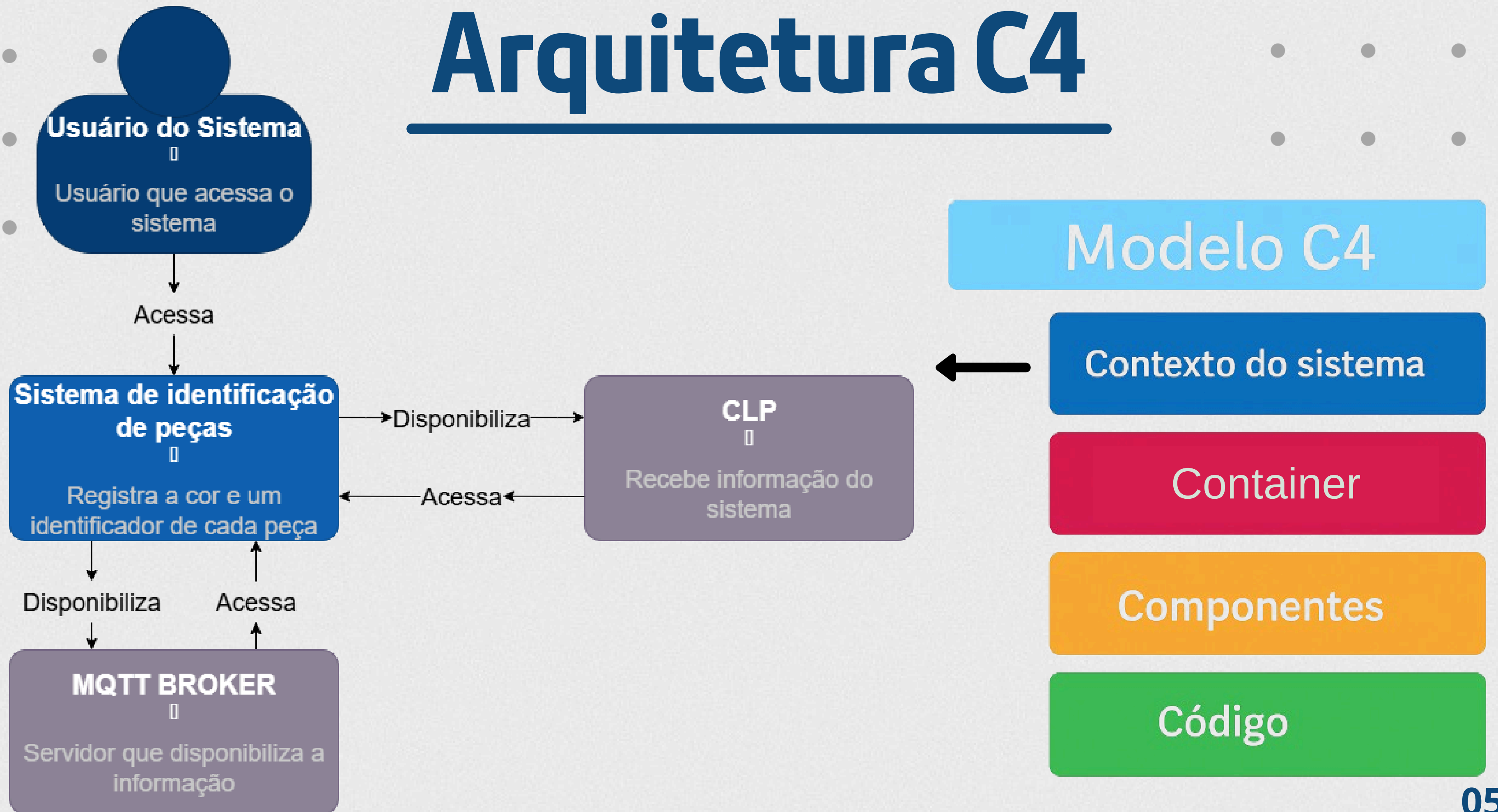


Configuração do Raspberry Pi

Instalação do Virtual Env E OpenCV

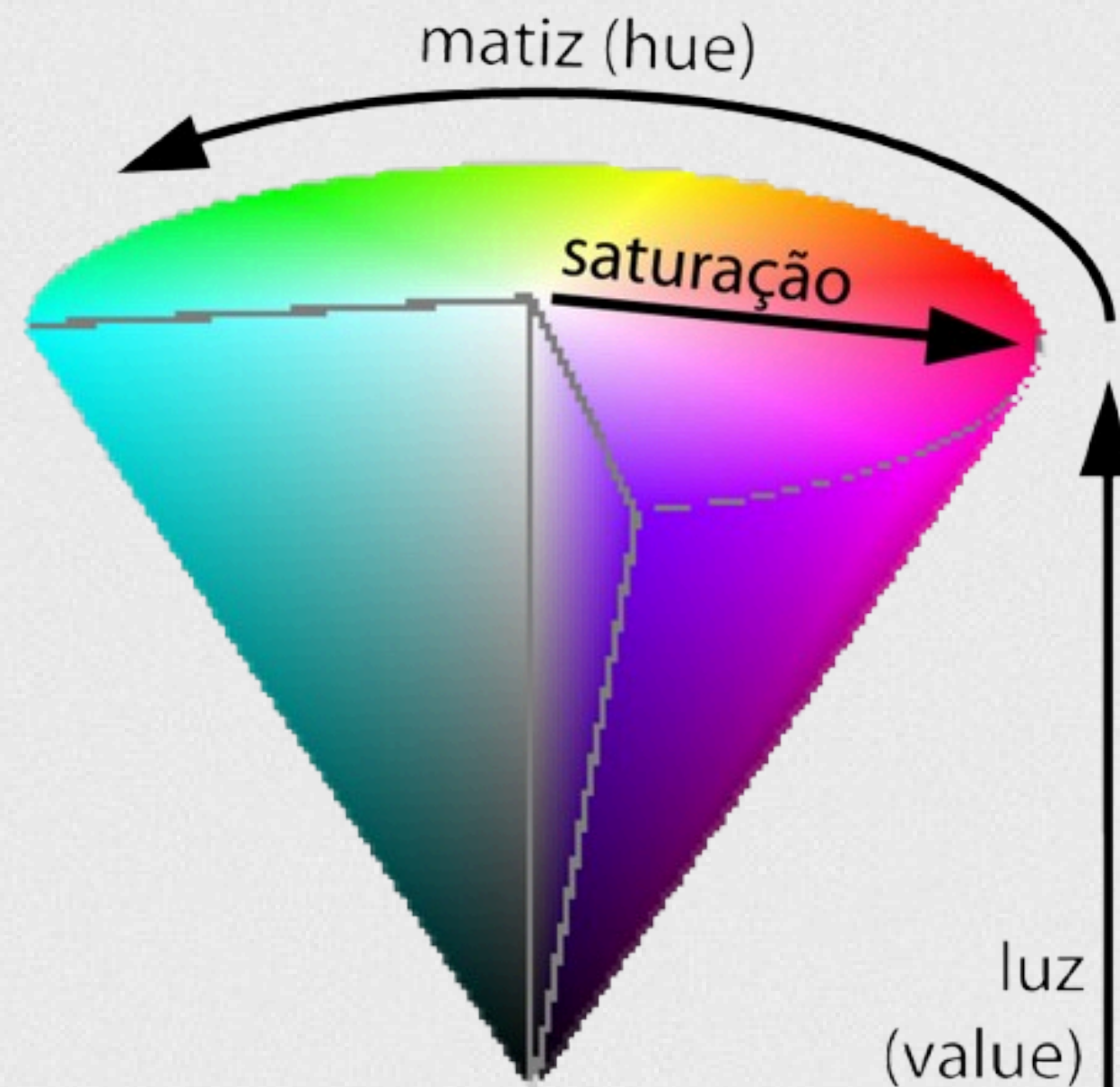


Arquitetura C4



Visão Computacional

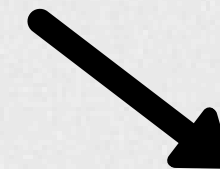
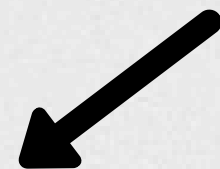
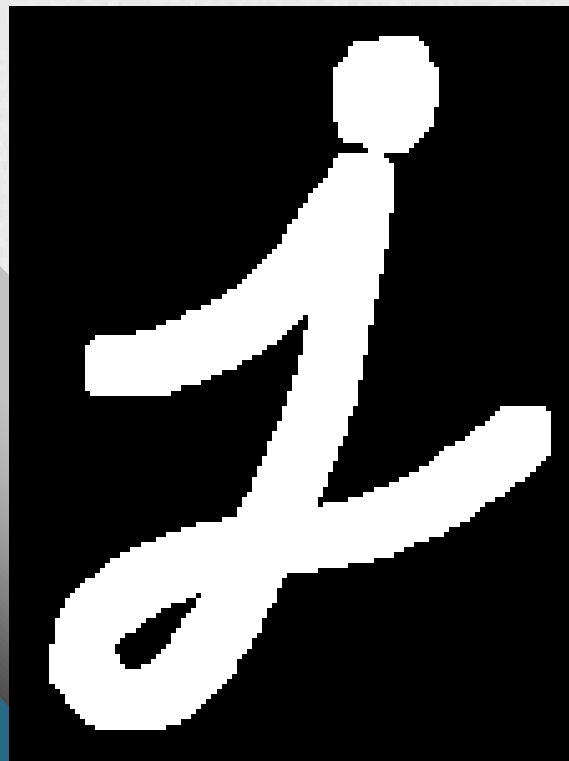
Espaço HSV



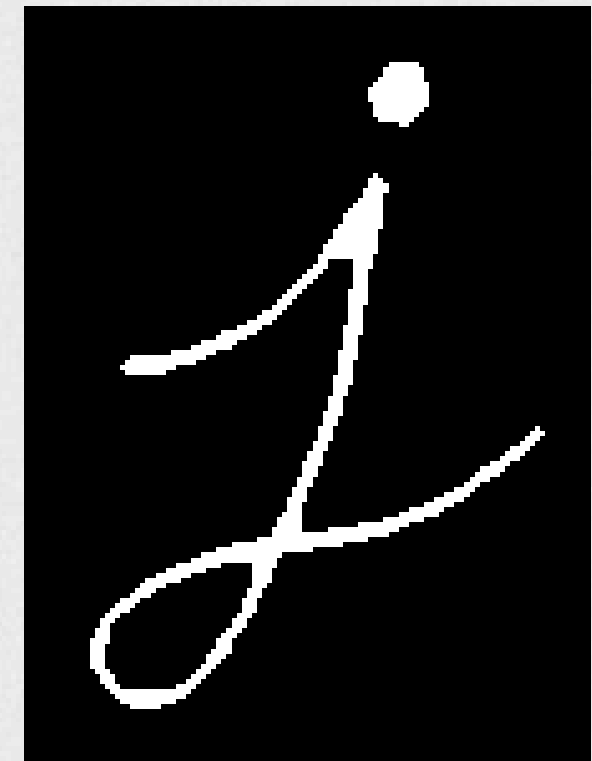
Visão Computacional

Operações Morfológicas

Dilatação

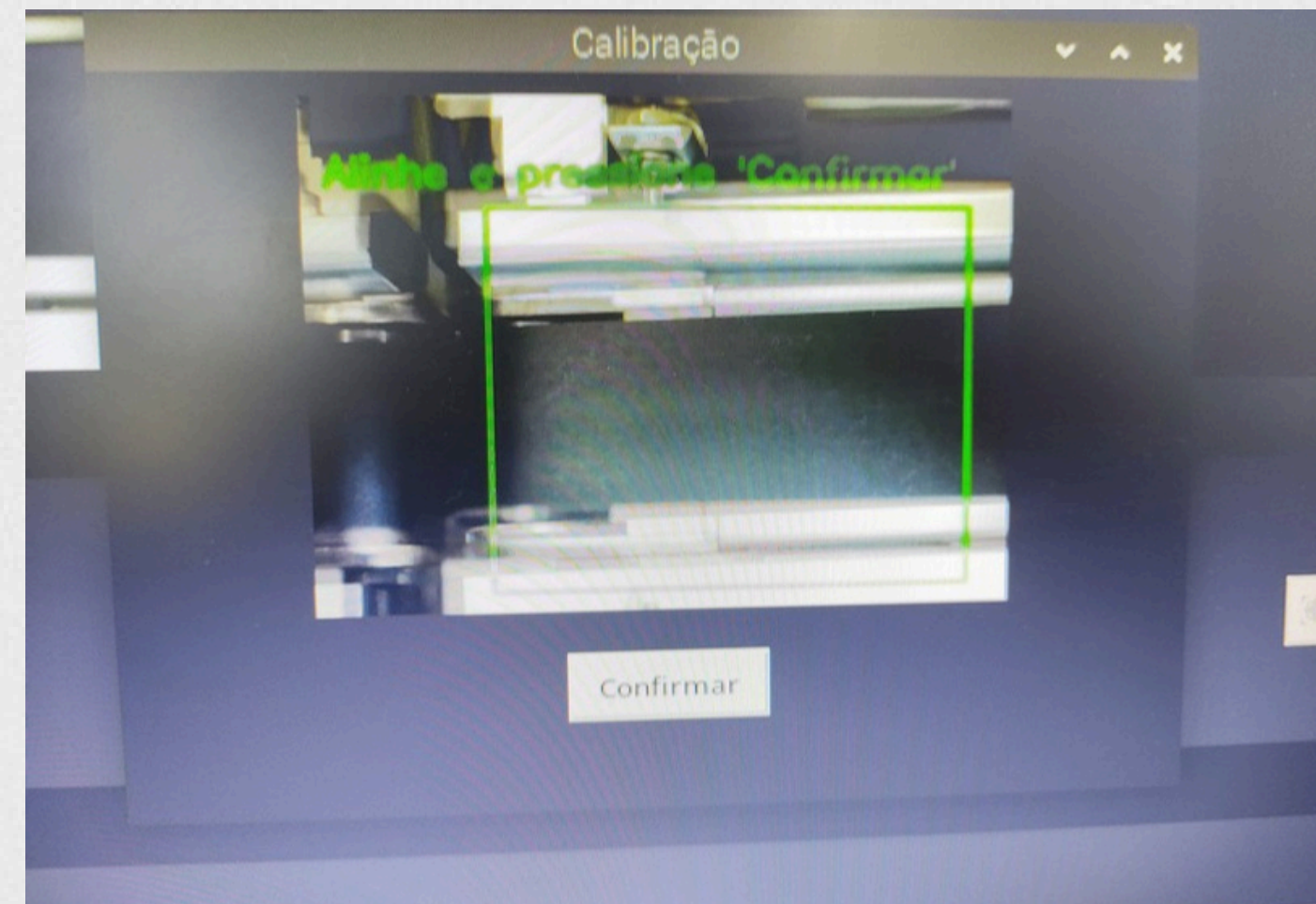
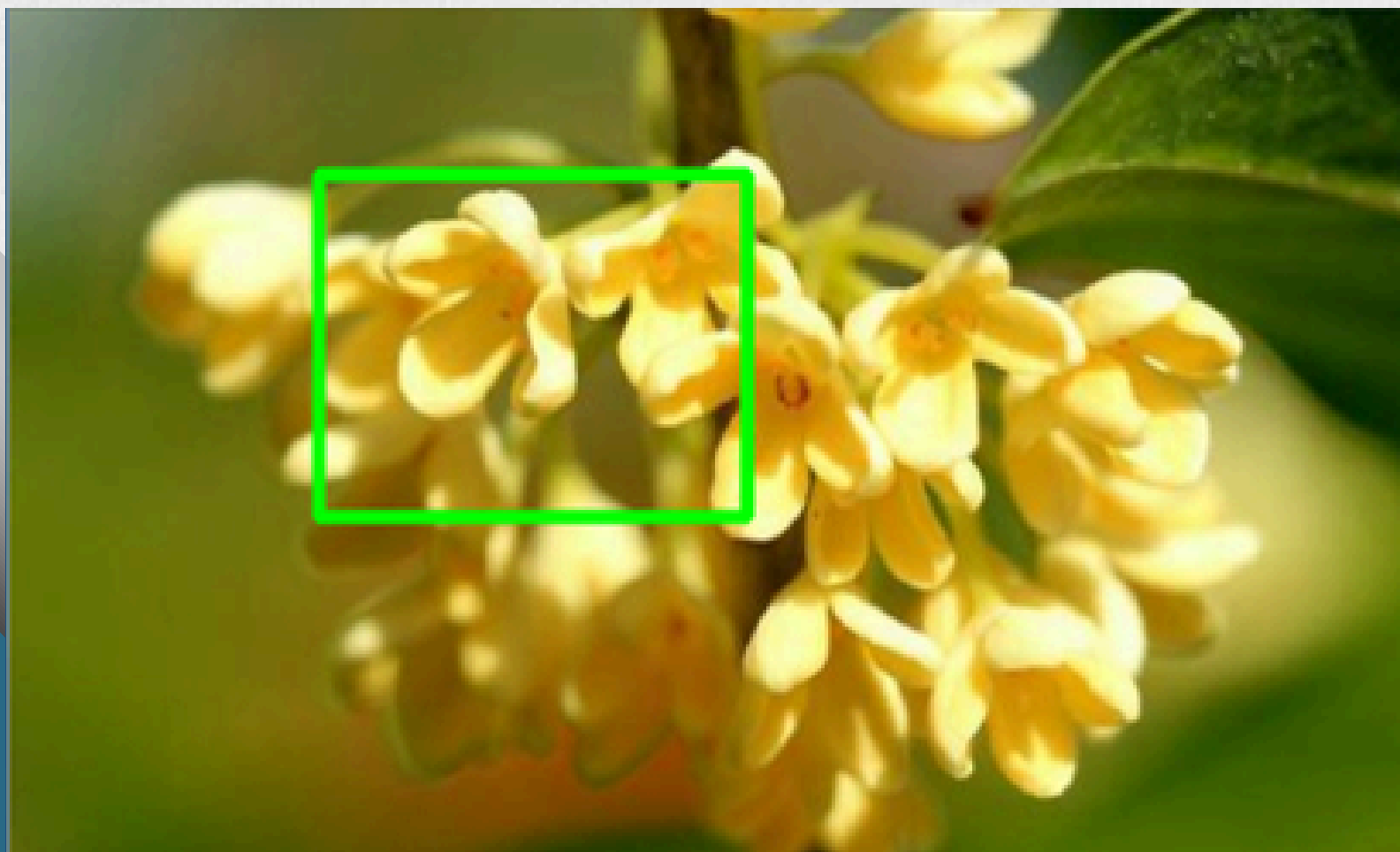


Erosão



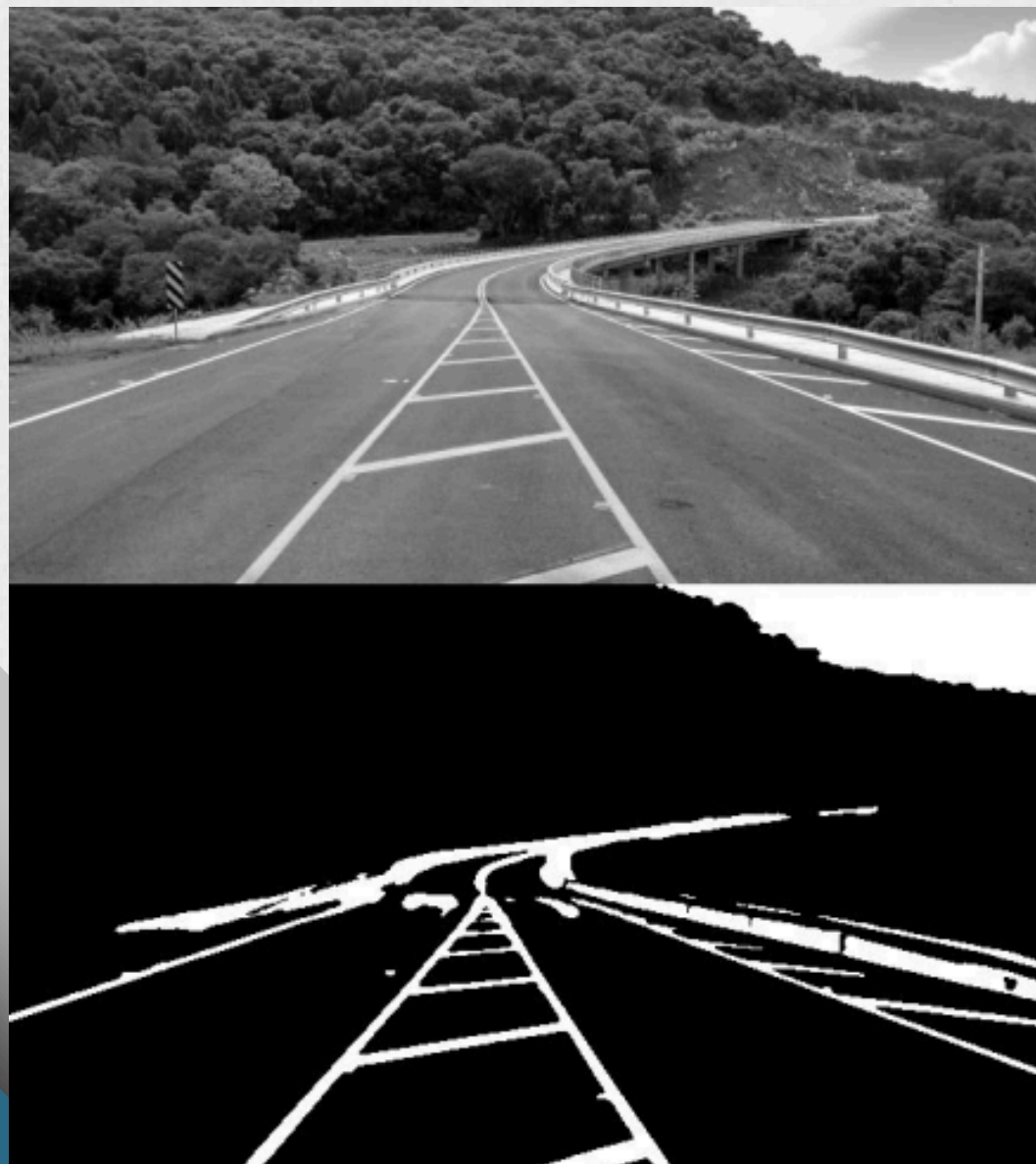
Visão Computacional

ROI - *Region of Interest*



Visão Computacional

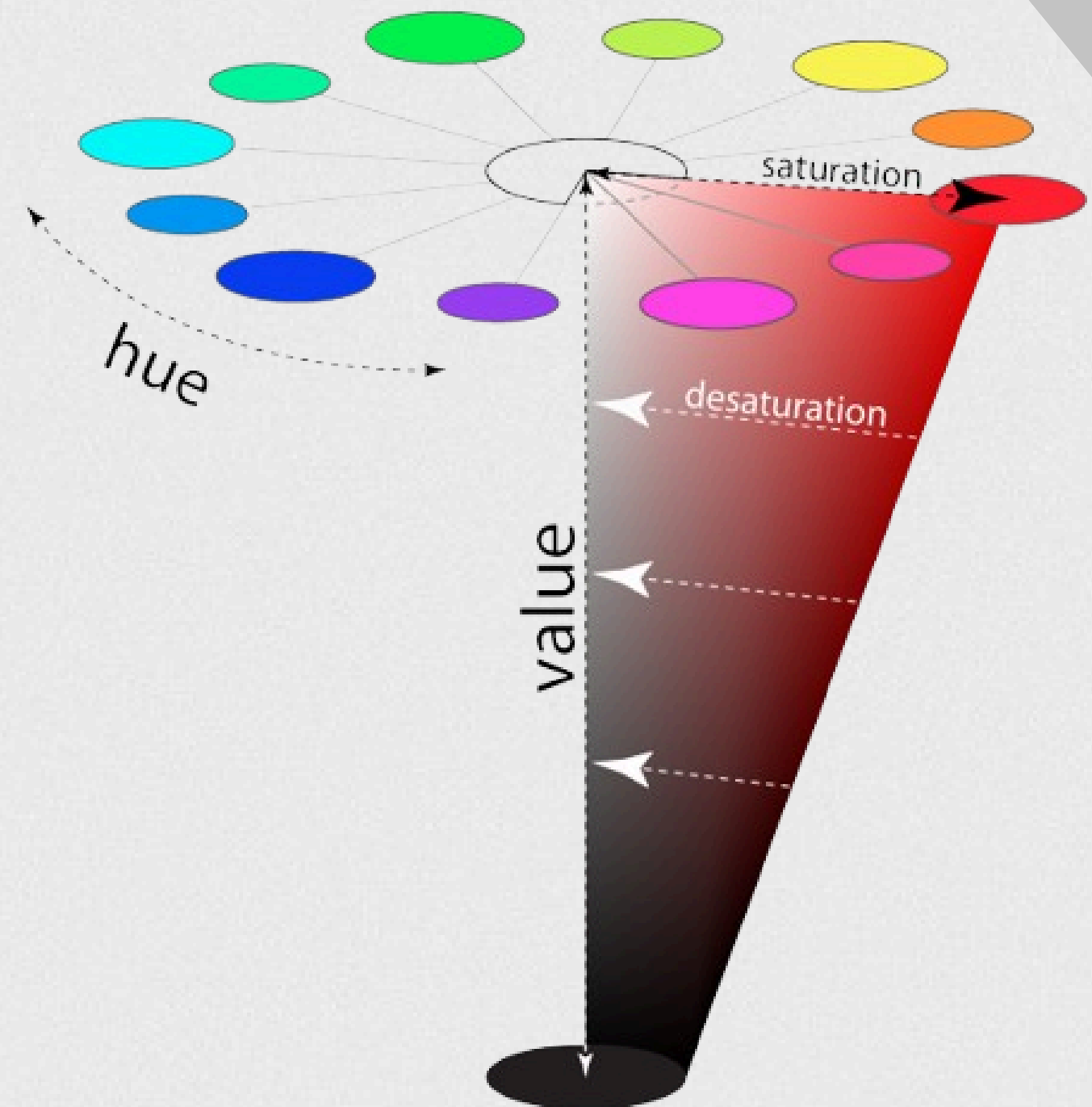
Binarização com Limiar



Converter uma imagem em tons de cinza para preto e branco, com base em um valor de intensidade (thresholding).

Identificação de Cores - Vermelho

- Define-se limites no espaço hsv.
- Dilatação para reforçar contornos.
- Filtragem dentro do ROI.

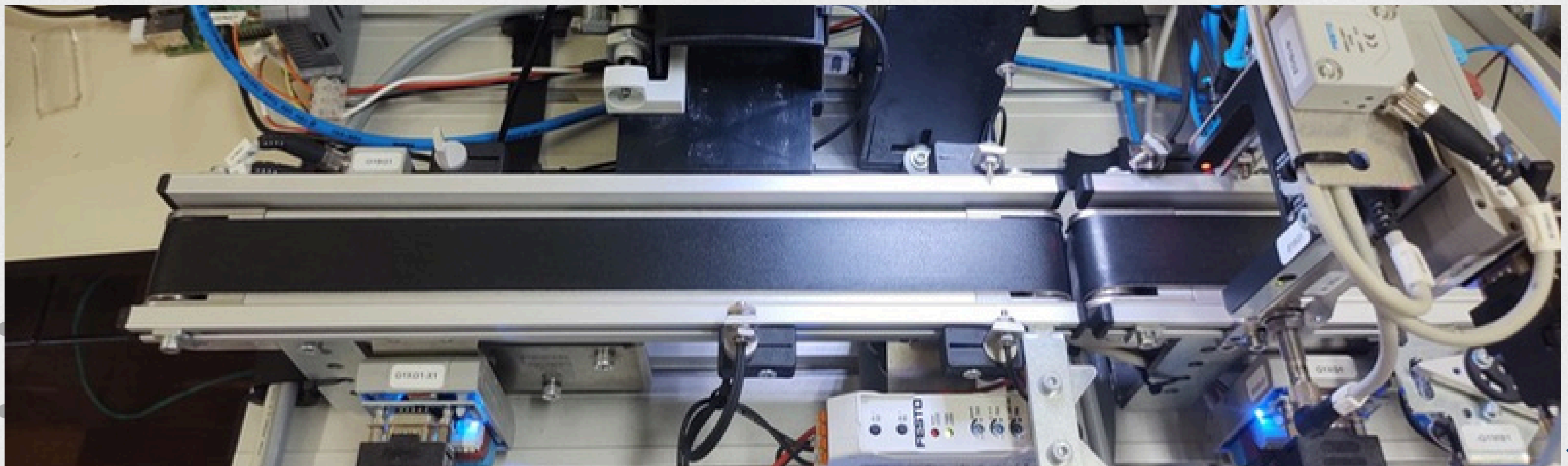


Identificação de Cores - Vermelho



Identificação de Cores - Preto

Desafio pois a esteira é preta, não possibilitando segmentação direta por cor.



Identificação de Cores - Preto

Solução: Subtração de fundo



—



=



Identificação de Cores - Preto

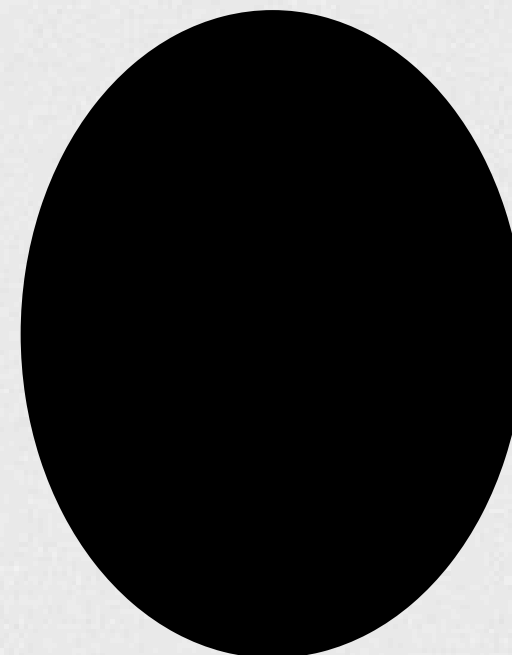
Imagem em Movimento



Limiarização
com threshold
preto: **40**



Imagem em Movimento
Binarizada

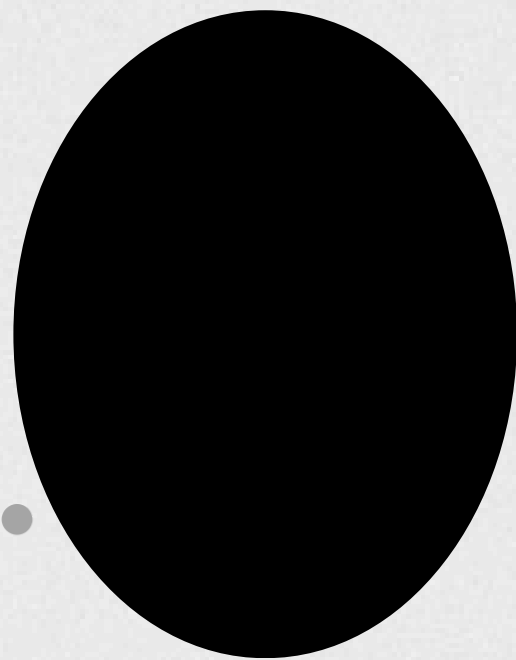


Assim obtemos
as partes pretas
da imagem em
movimento.

Identificação de Cores - Preto

Comparação da imagem binarizada com a máscara HSV para pretos

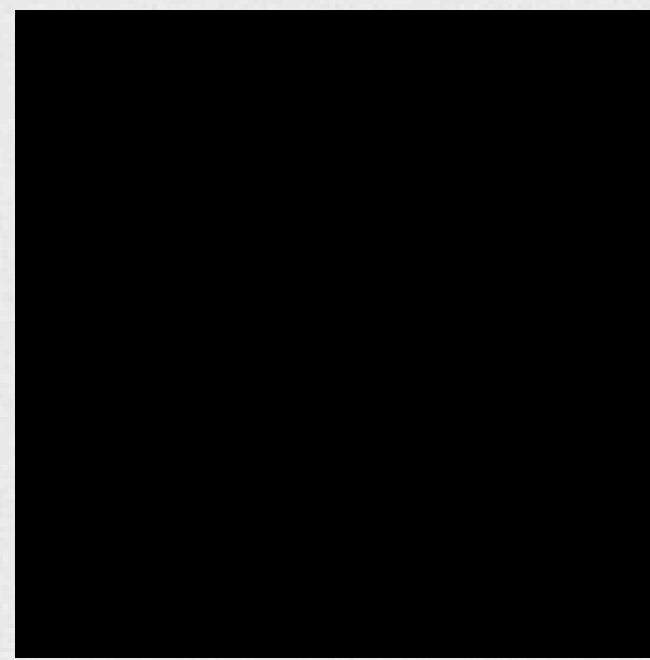
Imagem em Movimento
Binarizada



`bitwise_and`

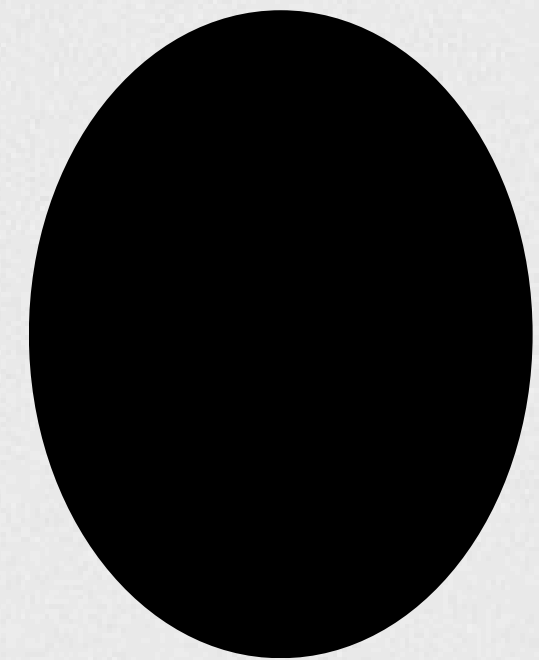
`&`

Máscara HSV Limitada a Faixa de
Pretos



`=`

Imagem Preta em
Movimento

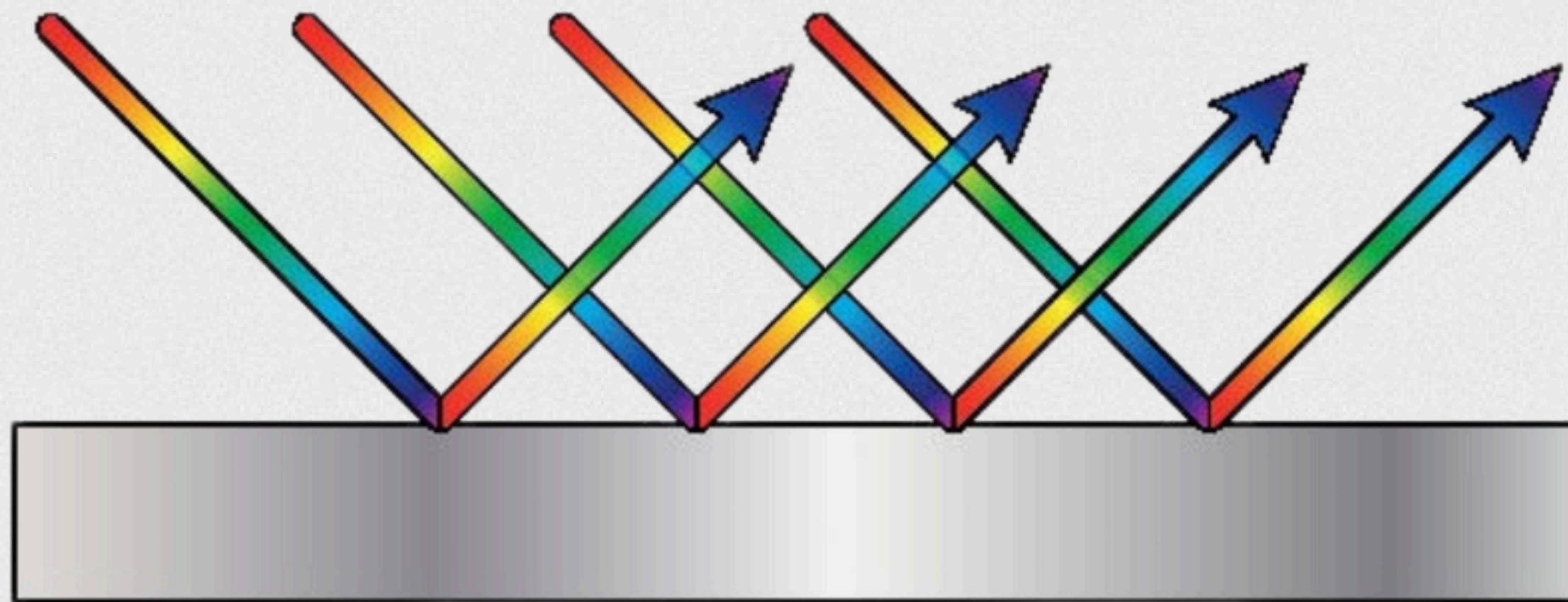


Identificação de Cores - Preto



Identificação de Cores - Cromado

Desafio pois o brilho da peça é alto e a reflexividade também.



Identificação de Cores - Cromado

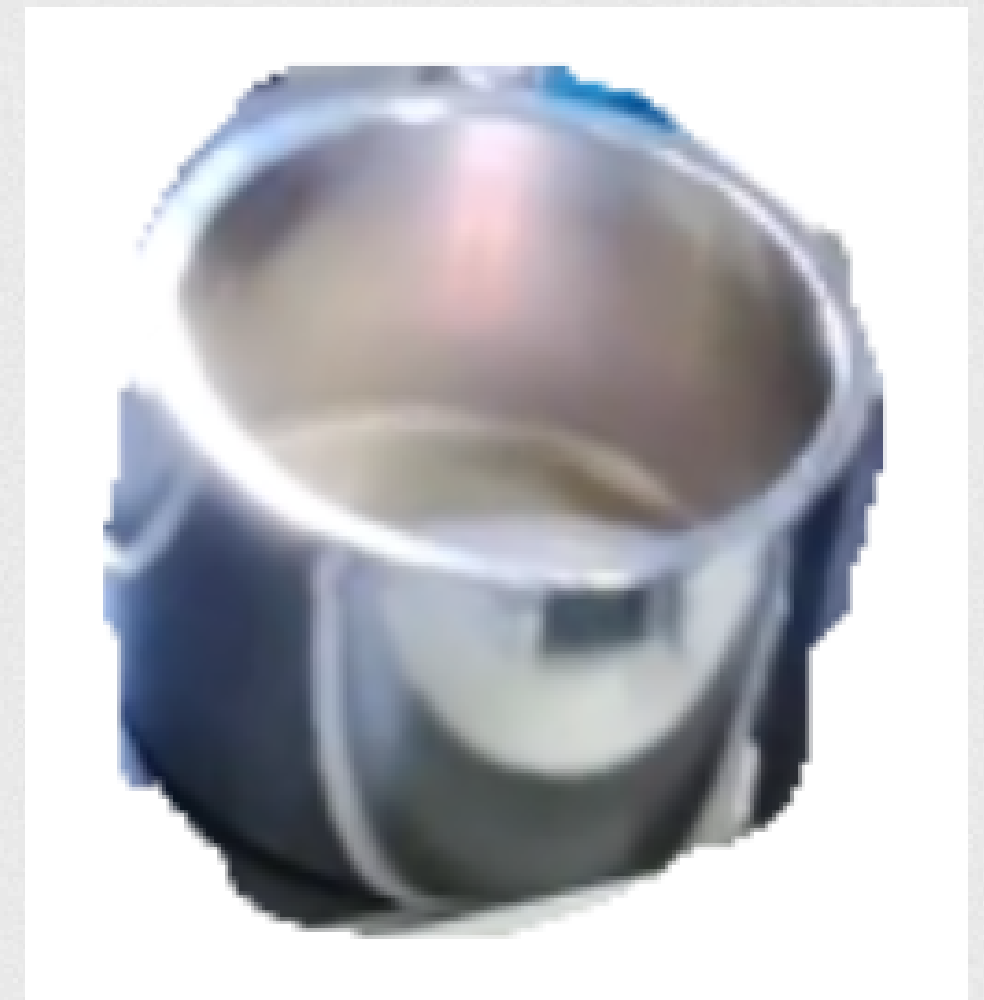
Solução: Subtração de fundo



—



=



Identificação de Cores - Cromado

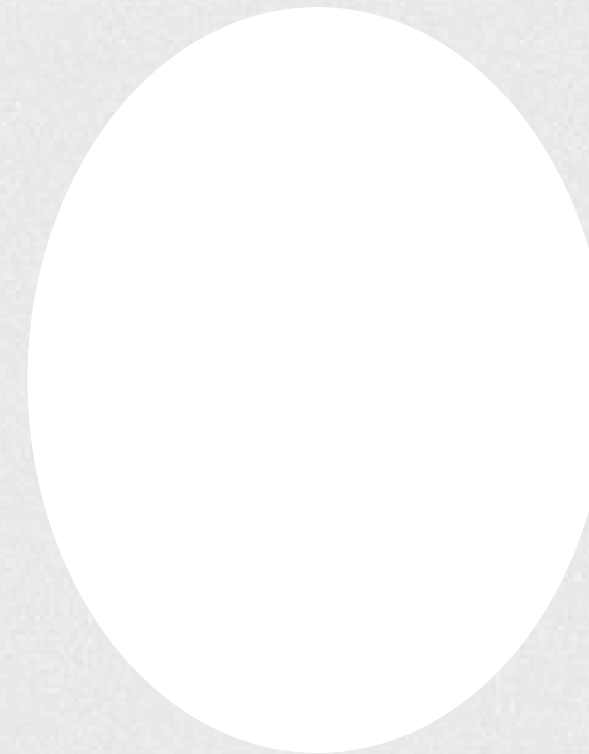
Imagem em Movimento



Limiarização
com threshold
cromado: **20**



Imagem em Movimento
Binarizada

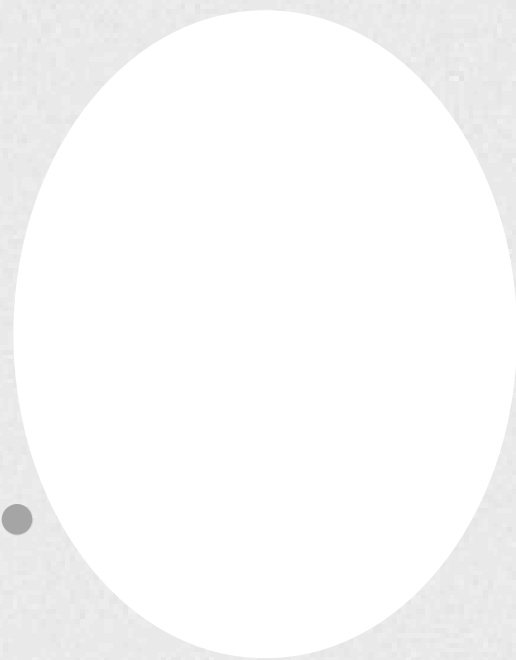


Assim obtemos as
partes altamente
reflexivas da imagem
em movimento.

Identificação de Cores - Cromado

Comparação da imagem binarizada com a máscara HSV para baixo nível de saturação e alto valor de brilho.

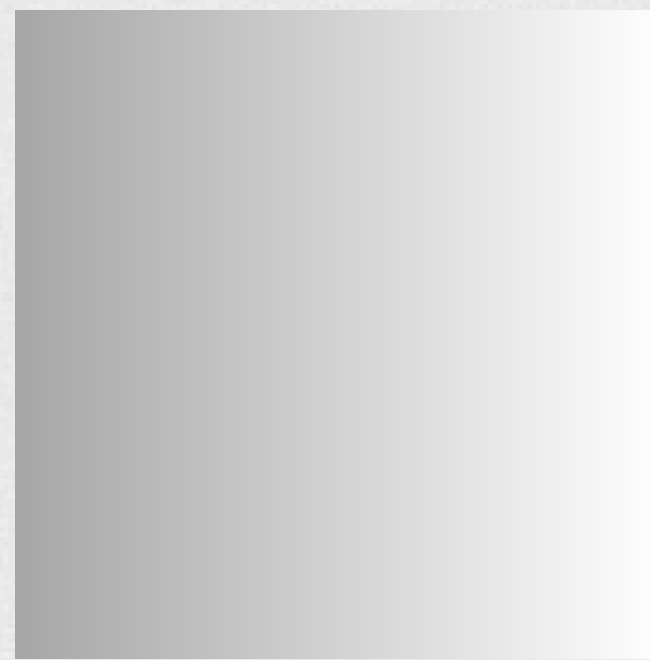
Imagem em Movimento
Binarizada



bitwise_and

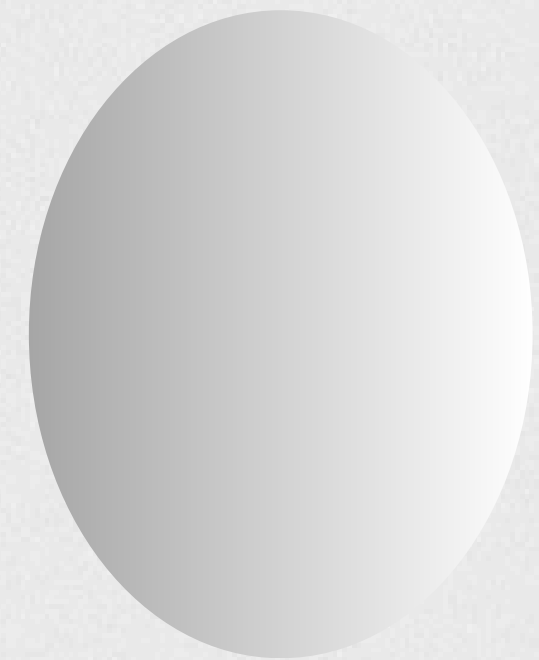
&

Máscara HSV “Cromada”

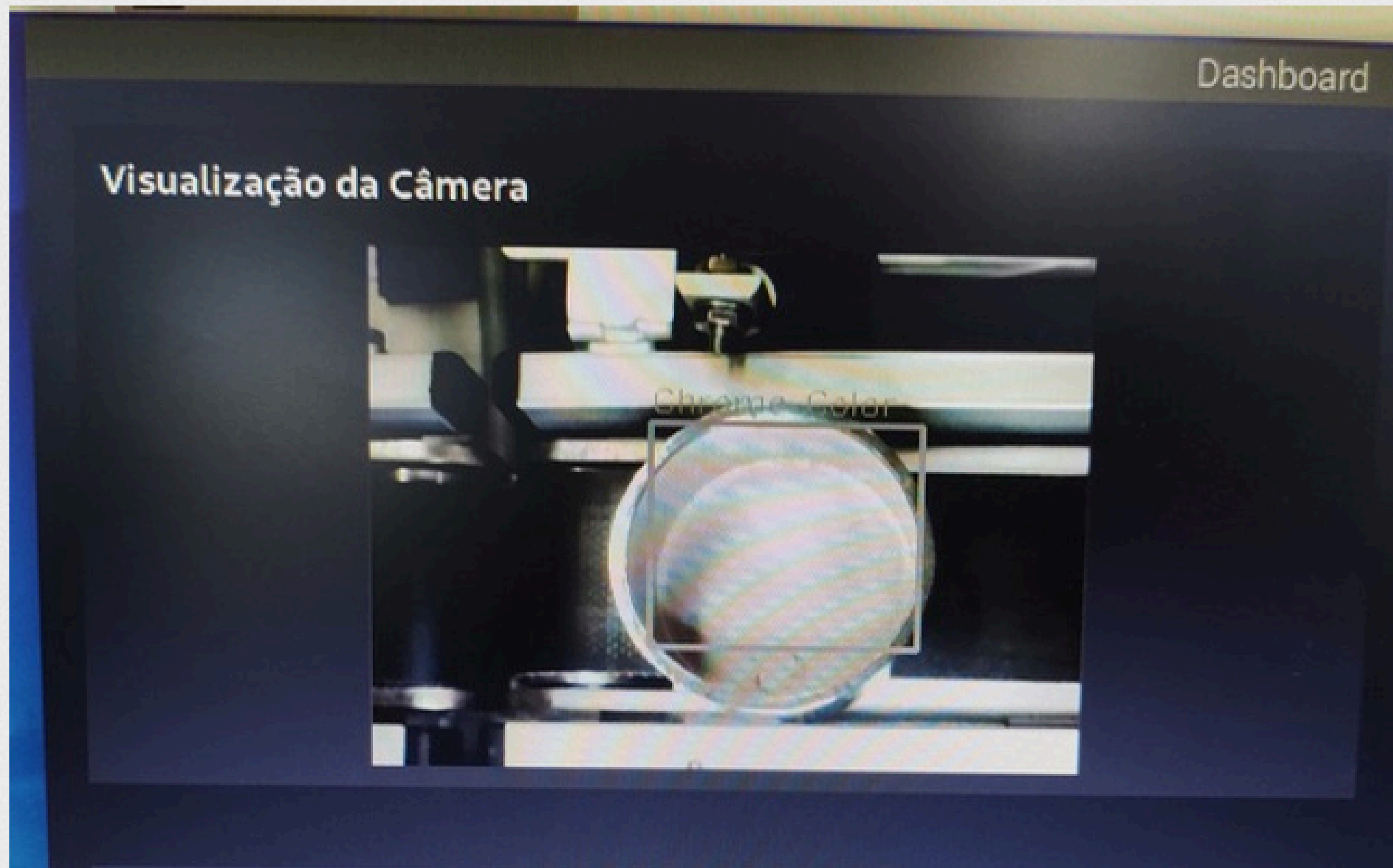


=

Imagem Cromada em
Movimento



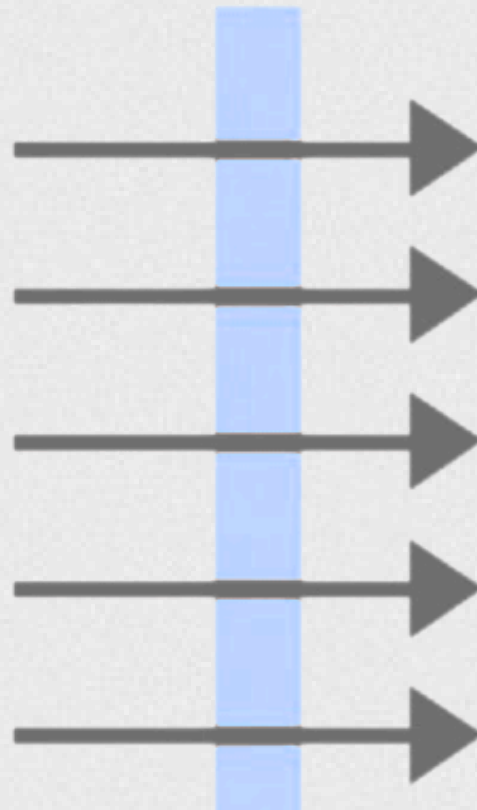
Identificação de Cores - Cromado



Identificação de Peças - Transparente

Desafio pois transparente não é uma cor definida, esteira continua parcialmente visível através dela.

Inviabiliza o uso de máscaras no espaço HSV.



Identificação de Peças - Transparente

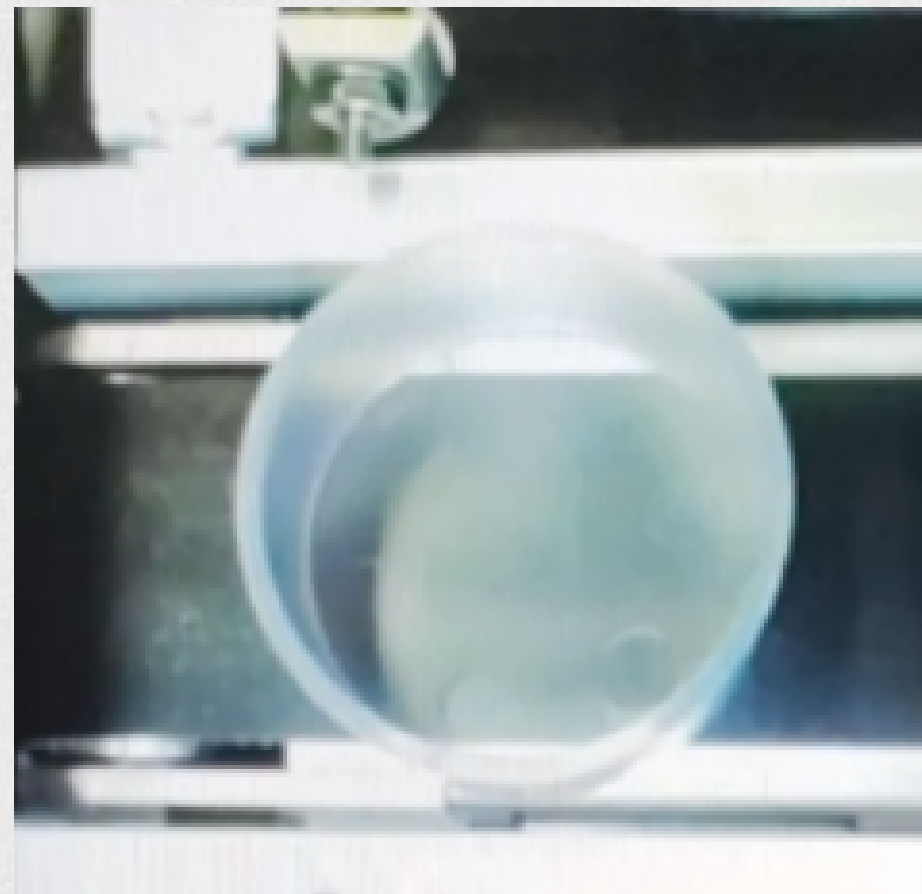
Solução: Exclusivamente subtração de fundo, agora com uma sensibilidade maior

Imagem Referência



—

Imagem Fundo



=

Imagem em Movimento



Identificação de Peças - Transparente

Imagem em Movimento



Limiarização
com threshold
transparente:

30



Imagem em Movimento
Binarizada



Assim ob.temos pequenas variações de intensidade luminosas provocadas ao distorcer levemente o fundo estático, principalmente pelas bordas e espessura do material.

Identificação de Peças - Transparente

Operações Morfológicas

Erosão:

- Remover pequenos ruídos (falsas detecções).
- Suaviza a imagem, preservando apenas regiões consistentes.

Dilatação:

- Reforça o contorno e expande as regiões brancas.
- Facilita a detecção de contornos.

Identificação de Peças - Transparente

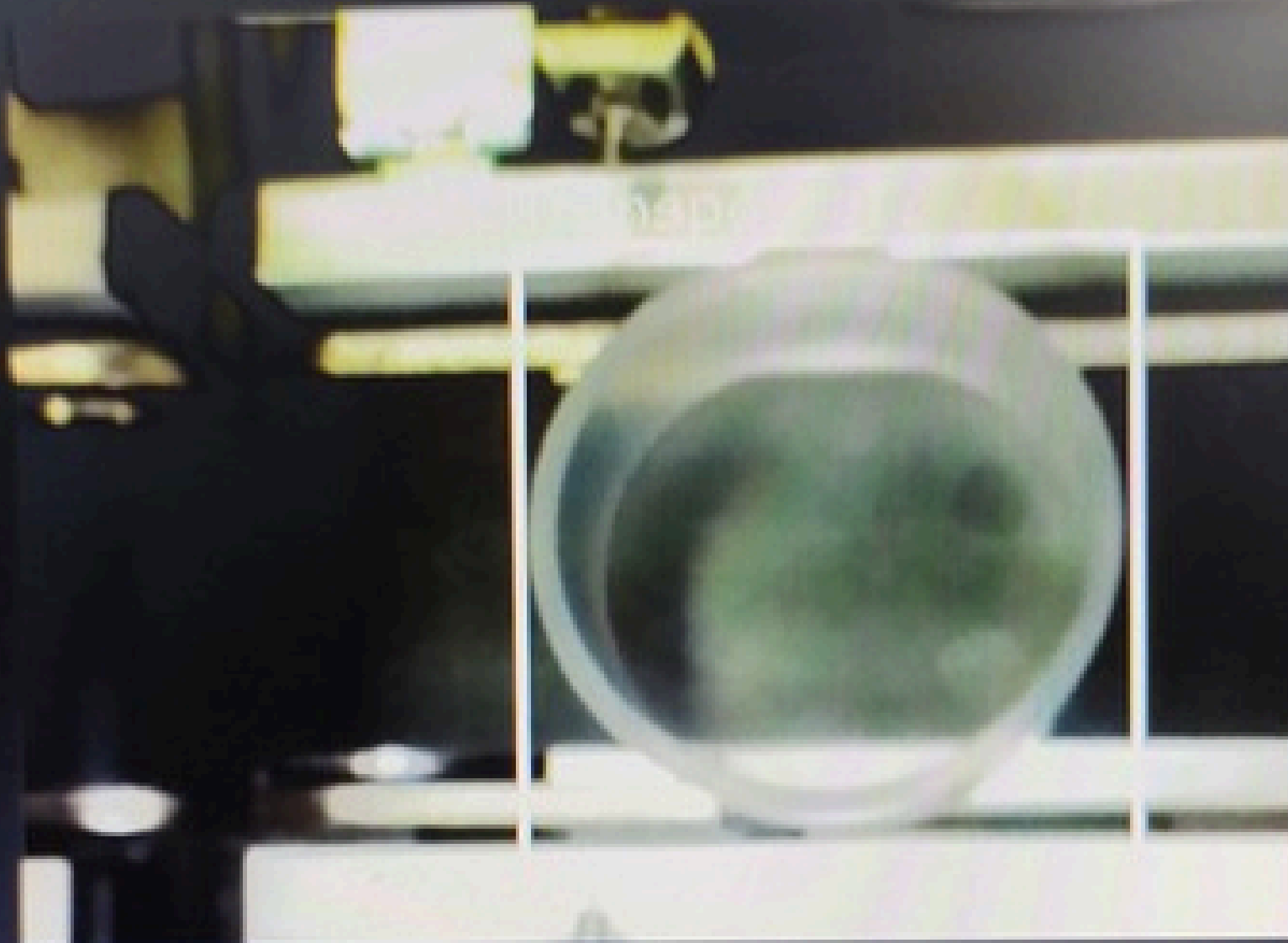
Processamento após as operações morfológicas

Peça transparente
em movimento



Identificação de Peças - Transparente

Visualização da Câmera

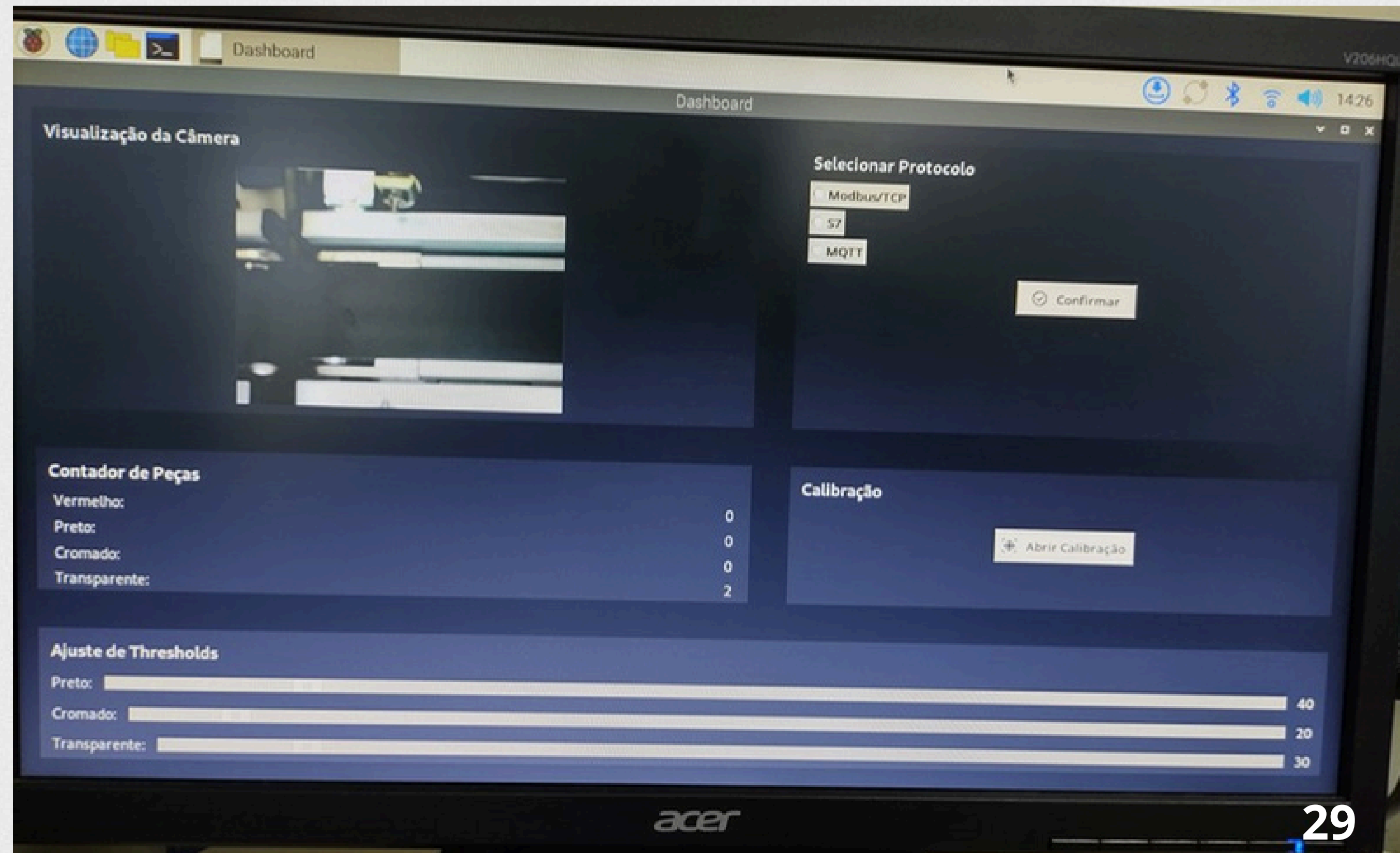


Calibração



Interface

Biblioteca:



Protocolos de Comunicação

- Modbus TCP
- MQTT
- S7

ModBus TCP

- Protocolo Mestre-Escravo:

O cliente (mestre) envia requisições e o servidor (escravo) responde. Muito usado em CLPs e sensores industriais.

- Comunicação via Ethernet:

Utiliza o protocolo TCP/IP na porta 502, facilitando a integração com redes modernas.

ModBus TCP

- Leitura e Escrita em Registros:
Acesso a entradas digitais, saídas e variáveis via "holding registers".
- Aplicações Industriais:
Controle de processos, supervisórios (SCADA) e monitoramento remoto de dispositivos.

Configuração TIA Portal

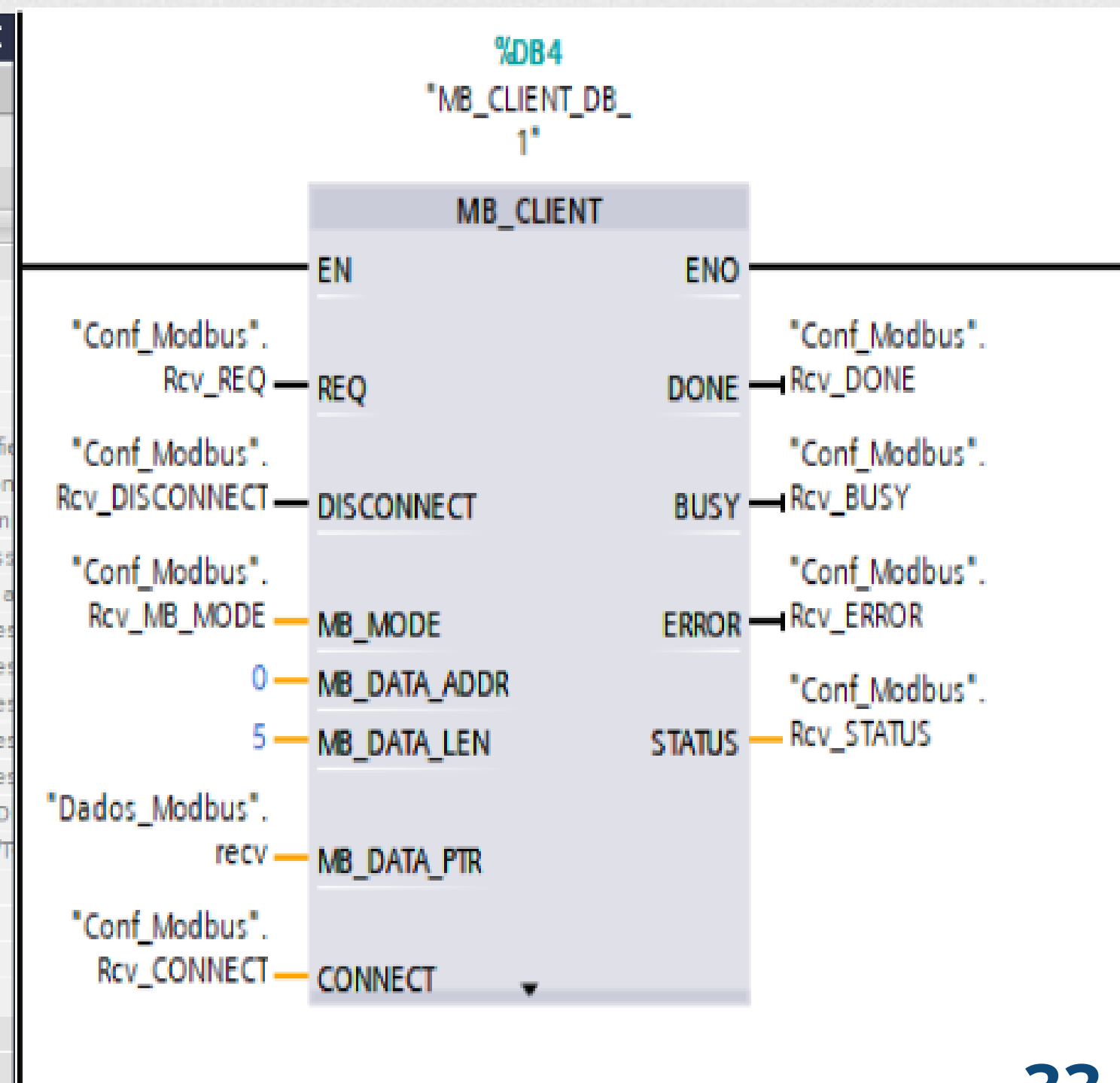
Project1 ▸ PLC_1 [CPU 1214C DC/DC/DC] ▸ Program blocks ▸ Data_block_1 [DB2]

Keep actual values Snapshot Copy snapshots to start values

Data_block_1

	Name	Data type	Start value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	Comment
1	Static			☐	☐	☐	☐	☐	
2	Rcv_REQ	Bool	false	☐	☒	☒	☒	☐	
3	Rcv_DISCONNECT	Bool	false	☐	☒	☒	☒	☐	
4	Rcv_MB_MODE	USInt	103	☐	☒	☒	☒	☐	
5	Rcv_CONNECT	TCON_IP_v4		☐	☒	☒	☒	☐	
6	InterfaceId	HW_ANY	64	☐	☒	☒	☒	☐	HWidentifi
7	ID	CONN_OUC	1	☐	☒	☒	☒	☐	connection
8	ConnectionType	Byte	11	☐	☒	☒	☒	☐	type of con
9	ActiveEstablished	Bool	1	☐	☒	☒	☒	☐	active/pass
10	RemoteAddress	IP_V4		☐	☒	☒	☒	☐	remote IP a
11	ADDR	Array[1..4] of Byte		☐	☒	☒	☒	☐	IPv4 addres
12	ADDR[1]	Byte	172	☐	☒	☒	☒	☐	IPv4 addres
13	ADDR[2]	Byte	16	☐	☒	☒	☒	☐	IPv4 addres
14	ADDR[3]	Byte	87	☐	☒	☒	☒	☐	IPv4 addres
15	ADDR[4]	Byte	189	☐	☒	☒	☒	☐	IPv4 addres
16	RemotePort	UInt	502	☐	☒	☒	☒	☐	remote UD
17	LocalPort	UInt	0	☐	☒	☒	☒	☐	local UDP/T
18	Rcv_DONE	Bool	false	☐	☒	☒	☒	☒	
19	Rcv_BUSY	Bool	false	☐	☒	☒	☒	☐	
20	Rcv_ERROR	Bool	false	☐	☒	☒	☒	☐	
21	Rcv_STATUS	Word	16#0	☐	☒	☒	☒	☐	
22	<Add new>			☐	☐	☐	☐	☐	

Configuração do ModBus/TCP



- **DATA_PTR:** Ponteiro para a área de memória no CLP onde os dados lidos do servidor serão armazenados, ou de onde serão lidos os dados a serem enviados. É necessário apontar para um bloco de dados (DB) válido e com o tamanho adequado à operação.

Configurações para iniciar um servidor ModBus TCP.

```
class ModbusTCP:
    def __init__(self):
        self.server = ModbusServer(host='172.16.87.189', port=502, no_block=True)
```

Envio das mensagens via protocolo ModBus TCP.

```
def comunicaModbusTCP(self, data):
    try:
        self.server.data_bank.set_holding_registers(0, [data["Vermelho"]])
        self.server.data_bank.set_holding_registers(1, [data["Preto"]])
        self.server.data_bank.set_holding_registers(2, [data["Cromado"]])
        self.server.data_bank.set_holding_registers(3, [data["Transparente"]])
        print("Registradores atualizados:", data)

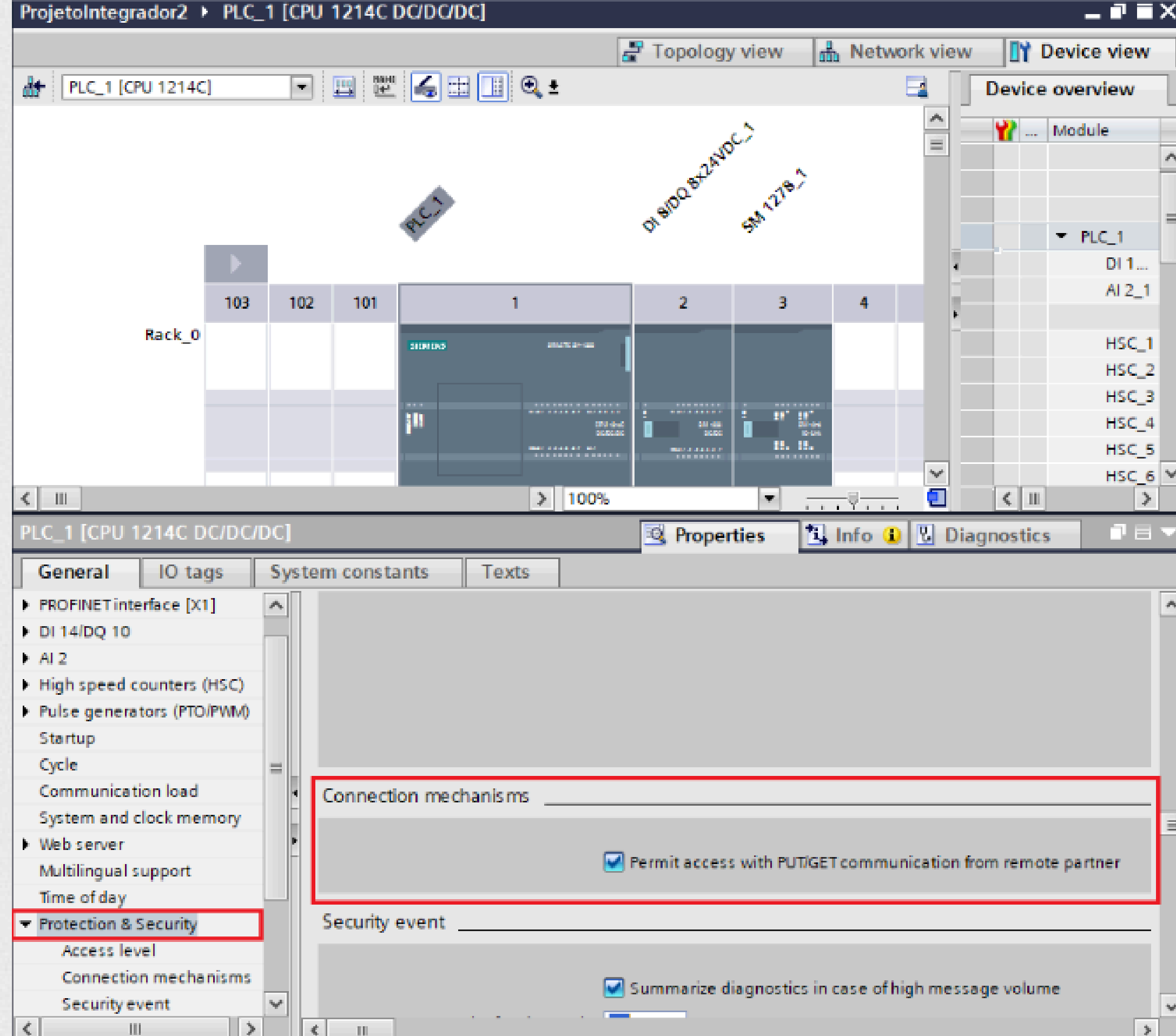
    except Exception as e:
        print("Erro ao escrever no Modbus:", e)
```

S7

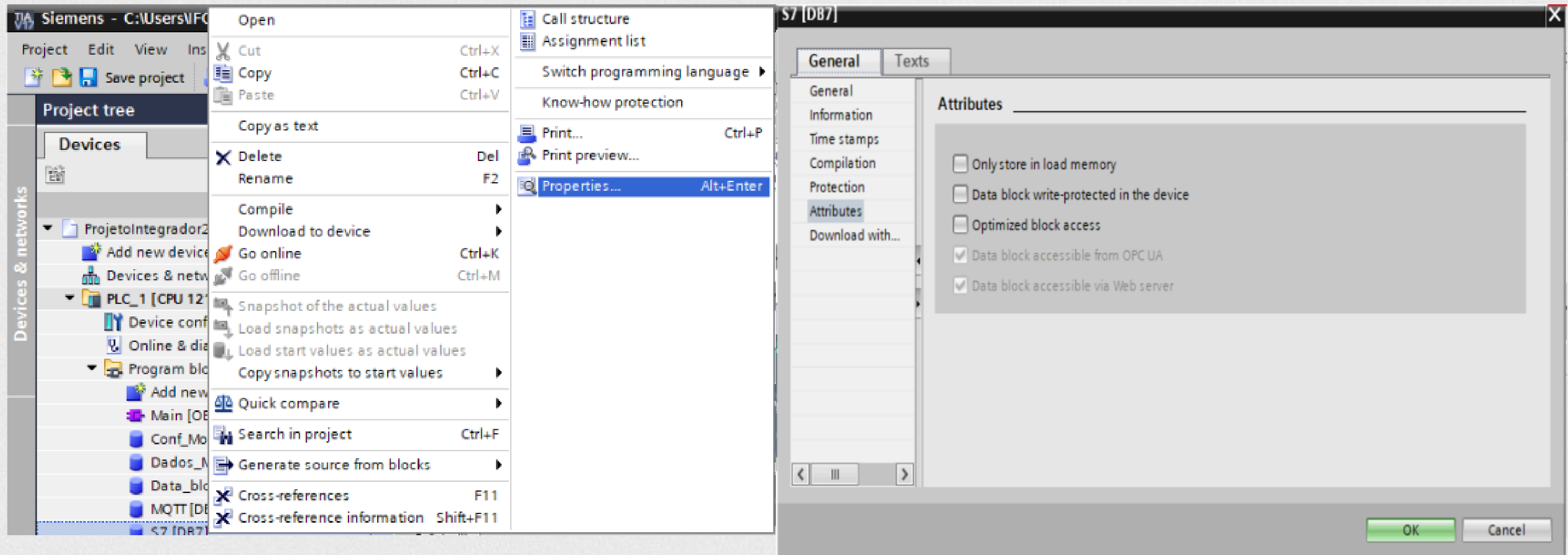
- Família S7 da Siemens:
CLPs como S7-1200 e S7-1500, amplamente utilizados na automação industrial.
- Comunicação via Profinet e S7 Protocol:
Utilizam Profinet nativamente e podem ser acessados por protocolos como S7 (snap7) e MODBUS TCP.

S7

- Programação no TIA Portal:
Ferramenta oficial da Siemens para criação de lógicas, configuração de redes e diagnóstico.
- Integração com Sistemas Externos:
Comunicação com Python, SCADA ou IoT via DBs e protocolos industriais.



Ativar as opções para usar protocolo S7.



S7										
		Name	Data type	Offset	Start value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint
1	Static					☐	☐	☐	☐	☐
2	Vermelho	Int	0.0	0	☐	☒	☒	☒	☒	☐
3	Preto	Int	2.0	0	☐	☒	☒	☒	☒	☐
4	Cromado	Int	4.0	0	☐	☒	☒	☒	☒	☐
5	Transparente	Int	6.0	0	☐	☒	☒	☒	☒	☐
5	Sinal_atualizacao	Bool	8.0	false	☐	☒	☒	☒	☒	☐

Essa é a DB utilizada para a comunicação via protocolo S7. O diferencial desse método é o uso de um **offset**, que permite escrever diretamente na posição correta da DB do CLP, conforme a cor da peça identificada no sistema. Esse mesmo processo ocorre também na comunicação via **MQTT** — a principal diferença está em **qual DB é utilizada**: no caso do S7, os dados são escritos na **DB7**, enquanto no MQTT são armazenados na **DB3**.

Configurações para iniciar comunicação s7.

```
class S7Client:
    def __init__(self, ip='172.16.87.61', rack=0, slot=1, db_number=7):
        self.client = snap7.client.Client()
        self.ip = ip
        self.rack = rack
        self.slot = slot
        self.db_number = db_number
        self.db_size = 16 #ajustar valor real
        self.db = None
```

```
def comunicaS7(self, data):
    try:
        self.db = self.client.db_read(self.db_number, 0, 9)

        set_int(self.db, 0, data["Vermelho"])
        set_int(self.db, 2, data["Preto"])
        set_int(self.db, 4, data["Cromado"])
        set_int(self.db, 6, data["Transparente"])

        set_bool(self.db, 8, 0, True)
        self.client.db_write(self.db_number, 0, self.db)

        print("Dados enviados ao CLP via S7: ", data)
    except Exception as e:
        print("Erro ao escrever no CLP:", e)
```

MQTT

- Modelo Publish/Subscribe:

Dispositivos publicam ou assinam mensagens em tópicos. Não se comunicam diretamente entre si.

- Uso de Broker:

O broker central (como Mosquitto) gerencia a entrega das mensagens entre os dispositivos conectados.

Como o CLP utilizado não possui suporte nativo ao protocolo MQTT, optamos por enviar as mensagens ao broker via MQTT e, paralelamente, transmitir as informações ao CLP utilizando o protocolo S7. Os mesmos procedimentos de escrita foram aplicados na DB do CLP, seguindo a lógica utilizada na comunicação via S7.

MQTT										
		Name	Data type	Offset	Start value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint
1		Static				☐	☐	☐	☐	☐
2		Vermelho	Int	0.0	0	☐	☒	☒	☒	☐
3		Preto	Int	2.0	0	☐	☒	☒	☒	☐
4		Cromado	Int	4.0	0	☐	☒	☒	☒	☐
5		Transparente	Int	6.0	0	☐	☒	☒	☒	☐
6		Sinal_atualizacao	Bool	8.0	false	☐	☒	☒	☒	☐

Configurações para iniciar enviar mensagens via Broker e iniciar comunicação s7 para enviar dados ao CLP.

```
class MQTT:
    def __init__(self, ip='172.16.87.61', rack=0, slot=1, db_number=3):

        self.client = mqtt.Client()
        self.client_s7 = snap7.client.Client()
        self.ip = ip
        self.rack = rack
        self.slot = slot
        self.db_number = db_number
        self.db = None
        self.db_size = 16 #ajustar valor real

        self.broker = "191.52.39.88"
        self.port = 1883
        self.topic = "projeto/cores"
```

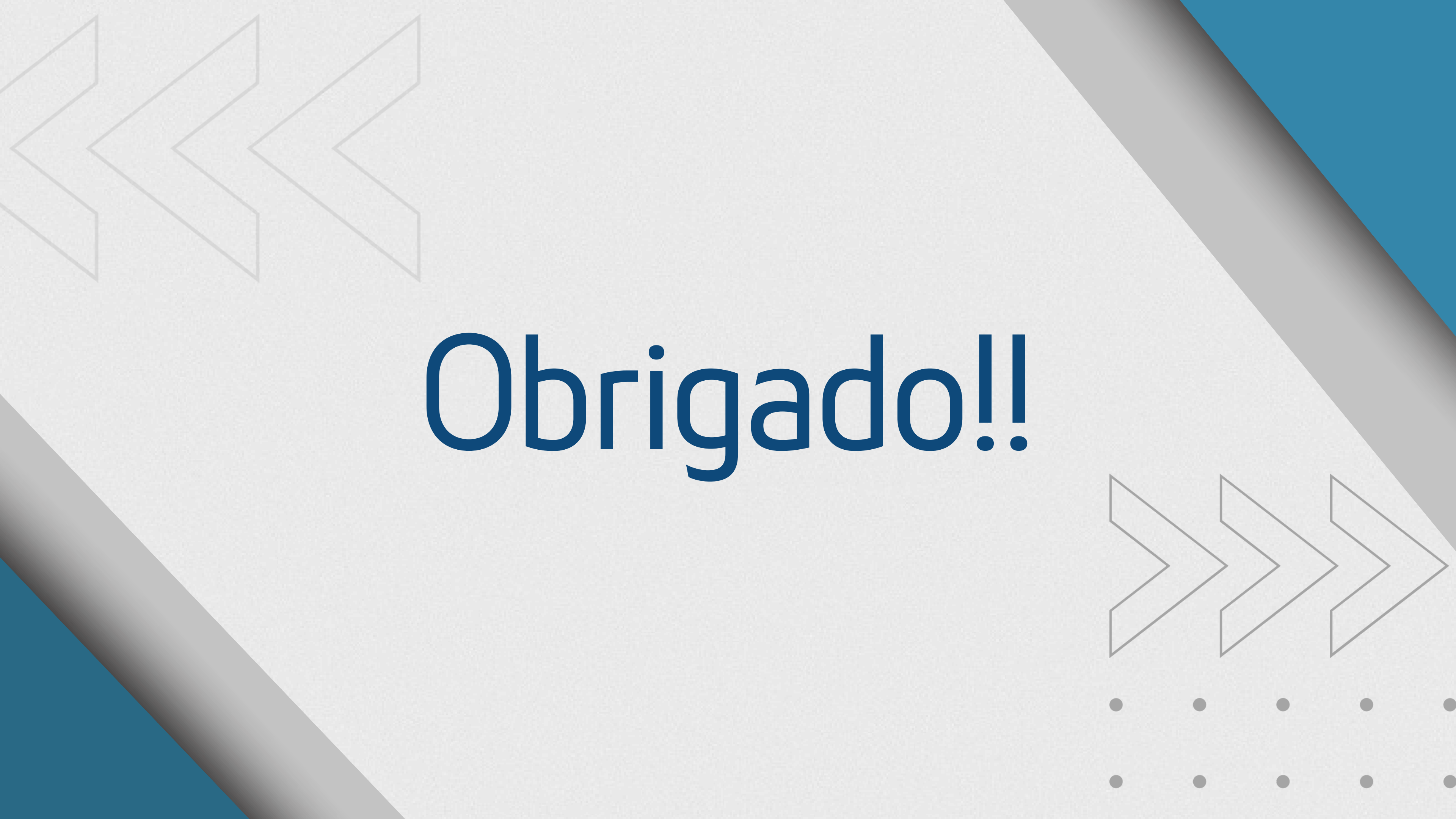

Envio das mensagens via MQTT.

```
def comunicaMQTT(self, data):
    try:
        payload = str(data)
        self.client.publish(self.topic, payload)
        print("MQTT publicou:", payload)

        self.db = self.client_s7.db_read(self.db_number, 0, 9)
        set_int(self.db, 0, data["Vermelho"])
        set_int(self.db, 2, data["Preto"])
        set_int(self.db, 4, data["Cromado"])
        set_int(self.db, 6, data["Transparente"])
        set_bool(self.db, 8, 0, True)
        self.client_s7.db_write(self.db_number, 0, self.db)
        #set_bool(self.db, 8, 0, True)
        print("Dados enviados ao CLP")
    except Exception as e:
        print("Erro ao publicar MQTT", e)
```

The background is a light gray with diagonal bands of darker gray and blue in the corners. In the top-left corner, there are three nested, light gray chevron arrows pointing left. In the bottom-right corner, there are three nested, light gray chevron arrows pointing right, and below them, a grid of ten small gray dots arranged in two rows of five.

Dúvidas?

The background features a light gray field with diagonal bands of blue and gray in the corners. In the top-left, three nested left-pointing chevrons are outlined in gray. In the bottom-right, three nested right-pointing chevrons are outlined in gray, with a 2x5 grid of small gray dots below them.

Obrigado!!